eda

October 3, 2024

[2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

[3]:
```
data = pd.read_csv("housing.csv")
```

[6]:
```
print("\nSample Data:")
print(data.head(10))
```

```
Sample Data:
   0.00632  18.00  2.310  0  0.5380  6.5750  65.20  4.0900  1  296.0  15.30  \
0  0.02731    0.0   7.07  0   0.469   6.421   78.9  4.9671  2  242.0   17.8
1  0.02729    0.0   7.07  0   0.469   7.185   61.1  4.9671  2  242.0   17.8
2  0.03237    0.0   2.18  0   0.458   6.998   45.8  6.0622  3  222.0   18.7
3  0.06905    0.0   2.18  0   0.458   7.147   54.2  6.0622  3  222.0   18.7
4  0.02985    0.0   2.18  0   0.458   6.430   58.7  6.0622  3  222.0   18.7
5  0.08829   12.5   7.87  0   0.524   6.012   66.6  5.5605  5  311.0   15.2
6  0.14455   12.5   7.87  0   0.524   6.172   96.1  5.9505  5  311.0   15.2
7  0.21124   12.5   7.87  0   0.524   5.631  100.0  6.0821  5  311.0   15.2
8  0.17004   12.5   7.87  0   0.524   6.004   85.9  6.5921  5  311.0   15.2
9  0.22489   12.5   7.87  0   0.524   6.377   94.3  6.3467  5  311.0   15.2

   396.90   4.98  24.00
0  396.90   9.14   21.6
1  392.83   4.03   34.7
2  394.63   2.94   33.4
3  396.90   5.33   36.2
4  394.12   5.21   28.7
5  395.60  12.43   22.9
6  396.90  19.15   27.1
7  386.63  29.93   16.5
8  386.71  17.10   18.9
9  392.52  20.45   15.0
```

[7]:
```
print("\nSample Data:")
print(data.tail(10))
```

```
Sample Data:
     0.00632  18.00  2.310  0  0.5380  6.5750  65.20  4.0900  1  296.0  15.30  \
495  0.28960    0.0   9.69  0   0.585   5.390   72.9  2.7986  6  391.0   19.2
496  0.26838    0.0   9.69  0   0.585   5.794   70.6  2.8927  6  391.0   19.2
497  0.23912    0.0   9.69  0   0.585   6.019   65.3  2.4091  6  391.0   19.2
498  0.17783    0.0   9.69  0   0.585   5.569   73.5  2.3999  6  391.0   19.2
499  0.22438    0.0   9.69  0   0.585   6.027   79.7  2.4982  6  391.0   19.2
500  0.06263    0.0  11.93  0   0.573   6.593   69.1  2.4786  1  273.0   21.0
501  0.04527    0.0  11.93  0   0.573   6.120   76.7  2.2875  1  273.0   21.0
502  0.06076    0.0  11.93  0   0.573   6.976   91.0  2.1675  1  273.0   21.0
503  0.10959    0.0  11.93  0   0.573   6.794   89.3  2.3889  1  273.0   21.0
504  0.04741    0.0  11.93  0   0.573   6.030   80.8  2.5050  1  273.0   21.0

     396.90   4.98  24.00
495  396.90  21.14   19.7
496  396.90  14.10   18.3
497  396.90  12.92   21.2
498  395.77  15.10   17.5
499  396.90  14.33   16.8
500  391.99   9.67   22.4
501  396.90   9.08   20.6
502  396.90   5.64   23.9
503  393.45   6.48   22.0
504  396.90   7.88   11.9
```

[8]:
```python
print("Basic information:")
print(data.info())
```

```
Basic information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   0.00632  505 non-null    float64
 1   18.00    505 non-null    float64
 2   2.310    505 non-null    float64
 3   0        505 non-null    int64
 4   0.5380   505 non-null    float64
 5   6.5750   505 non-null    float64
 6   65.20    505 non-null    float64
 7   4.0900   505 non-null    float64
 8   1        505 non-null    int64
 9   296.0    505 non-null    float64
 10  15.30    505 non-null    float64
 11  396.90   505 non-null    float64
 12  4.98     505 non-null    float64
```

```
 13  24.00    505 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.4 KB
None
```

[9]:
```
data.dtypes
```

[9]:
```
0.00632    float64
18.00      float64
2.310      float64
0            int64
0.5380     float64
6.5750     float64
65.20      float64
4.0900     float64
1            int64
296.0      float64
15.30      float64
396.90     float64
4.98       float64
24.00      float64
dtype: object
```

[10]:
```
data.columns
```

[10]:
```
Index(['0.00632', '18.00', '2.310', '0', '0.5380', '6.5750', '65.20', '4.0900',
       '1', '296.0', '15.30', '396.90', '4.98', '24.00'],
      dtype='object')
```

[11]:
```
data.shape
```

[11]:
```
(505, 14)
```

[12]:
```
print("\nSummary Statistics:")
print(data.describe())
```

```
Summary Statistics:
          0.00632       18.00       2.310           0      0.5380      6.5750  \
count  505.000000  505.000000  505.000000  505.000000  505.000000  505.000000
mean     3.620667   11.350495   11.154257    0.069307    0.554728    6.284059
std      8.608572   23.343704    6.855868    0.254227    0.115990    0.703195
min      0.009060    0.000000    0.460000    0.000000    0.385000    3.561000
25%      0.082210    0.000000    5.190000    0.000000    0.449000    5.885000
50%      0.259150    0.000000    9.690000    0.000000    0.538000    6.208000
75%      3.678220   12.500000   18.100000    0.000000    0.624000    6.625000
max     88.976200  100.000000   27.740000    1.000000    0.871000    8.780000

            65.20      4.0900           1       296.0       15.30      396.90  \
```

```
count  505.000000  505.000000  505.000000  505.000000  505.000000  505.000000
mean    68.581584    3.794459    9.566337  408.459406   18.461782  356.594376
std     28.176371    2.107757    8.707553  168.629992    2.162520   91.367787
min      2.900000    1.129600    1.000000  187.000000   12.600000    0.320000
25%     45.000000    2.100000    4.000000  279.000000   17.400000  375.330000
50%     77.700000    3.199200    5.000000  330.000000   19.100000  391.430000
75%     94.100000    5.211900   24.000000  666.000000   20.200000  396.210000
max    100.000000   12.126500   24.000000  711.000000   22.000000  396.900000

             4.98       24.00
count  505.000000  505.000000
mean    12.668257   22.529901
std      7.139950    9.205991
min      1.730000    5.000000
25%      7.010000   17.000000
50%     11.380000   21.200000
75%     16.960000   25.000000
max     37.970000   50.000000
```

[13]:
```
data.isnull().sum()
```

[13]:
```
0.00632    0
18.00      0
2.310      0
0          0
0.5380     0
6.5750     0
65.20      0
4.0900     0
1          0
296.0      0
15.30      0
396.90     0
4.98       0
24.00      0
dtype: int64
```

[14]:
```
data.dropna(inplace = True)
```

[15]:
```
data.count()
```

[15]:
```
0.00632    505
18.00      505
2.310      505
0          505
0.5380     505
6.5750     505
65.20      505
```

```
4.0900     505
1          505
296.0      505
15.30      505
396.90     505
4.98       505
24.00      505
dtype: int64
```

[16]:
```
data.dropna(inplace = True)
data.count()
```

[16]:
```
0.00632    505
18.00      505
2.310      505
0          505
0.5380     505
6.5750     505
65.20      505
4.0900     505
1          505
296.0      505
15.30      505
396.90     505
4.98       505
24.00      505
dtype: int64
```

[17]:
```
duplicate_rows_df = data[data.duplicated()]
print("number of duplicate rows:",duplicate_rows_df.shape)
```

```
number of duplicate rows: (0, 14)
```

[18]:
```
data.count()
```

[18]:
```
0.00632    505
18.00      505
2.310      505
0          505
0.5380     505
6.5750     505
65.20      505
4.0900     505
1          505
296.0      505
15.30      505
396.90     505
4.98       505
```

```
24.00      505
dtype: int64
```

[19]:
```
data = data.drop_duplicates()
```

[20]:
```
data.count()
```

[20]:
```
0.00632    505
18.00      505
2.310      505
0          505
0.5380     505
6.5750     505
65.20      505
4.0900     505
1          505
296.0      505
15.30      505
396.90     505
4.98       505
24.00      505
dtype: int64
```

[21]:
```
features=data.columns
features
```

[21]:
```
Index(['0.00632', '18.00', '2.310', '0', '0.5380', '6.5750', '65.20', '4.0900',
       '1', '296.0', '15.30', '396.90', '4.98', '24.00'],
      dtype='object')
```

[22]:
```
zero_val_cols=(data[features]==0).sum()
zero_val_cols
```

[22]:
```
0.00632      0
18.00      372
2.310        0
0          470
0.5380       0
6.5750       0
65.20        0
4.0900       0
1            0
296.0        0
15.30        0
396.90       0
4.98         0
24.00        0
```

```
dtype: int64
```

[23]:
```
data.isnull().sum()/len(data)*100
```

[23]:
```
0.00632    0.0
18.00      0.0
2.310      0.0
0          0.0
0.5380     0.0
6.5750     0.0
65.20      0.0
4.0900     0.0
1          0.0
296.0      0.0
15.30      0.0
396.90     0.0
4.98       0.0
24.00      0.0
dtype: float64
```

[28]:
```
data[['0.00632','18.00']] = data[['0.00632','18.00']].replace(0,np.NaN)
```

[30]:
```
data.18.00.fillna(data.18.00.median(),inplace=True)
```

```
  Cell In[30], line 1
    data.18.00.fillna(data.18.00.median(),inplace=True)
        ^

SyntaxError: invalid syntax
```

[33]:
```
one_hot_encoded = pd.get_dummies(data, columns=['6.5750'], prefix=['6.5750'])
print("One-Hot encoded data:")
print(one_hot_encoded)
```

```
One-Hot encoded data:
     0.00632  18.00  2.310  0  0.5380  65.20  4.0900  1  296.0  15.30  …  \
0    0.02731    NaN   7.07  0   0.469   78.9  4.9671  2  242.0   17.8  …
1    0.02729    NaN   7.07  0   0.469   61.1  4.9671  2  242.0   17.8  …
2    0.03237    NaN   2.18  0   0.458   45.8  6.0622  3  222.0   18.7  …
3    0.06905    NaN   2.18  0   0.458   54.2  6.0622  3  222.0   18.7  …
4    0.02985    NaN   2.18  0   0.458   58.7  6.0622  3  222.0   18.7  …
..       …    …     …  ..     …     …     …  ..     …     …  …
500  0.06263    NaN  11.93  0   0.573   69.1  2.4786  1  273.0   21.0  …
501  0.04527    NaN  11.93  0   0.573   76.7  2.2875  1  273.0   21.0  …
502  0.06076    NaN  11.93  0   0.573   91.0  2.1675  1  273.0   21.0  …
503  0.10959    NaN  11.93  0   0.573   89.3  2.3889  1  273.0   21.0  …
504  0.04741    NaN  11.93  0   0.573   80.8  2.5050  1  273.0   21.0  …
```

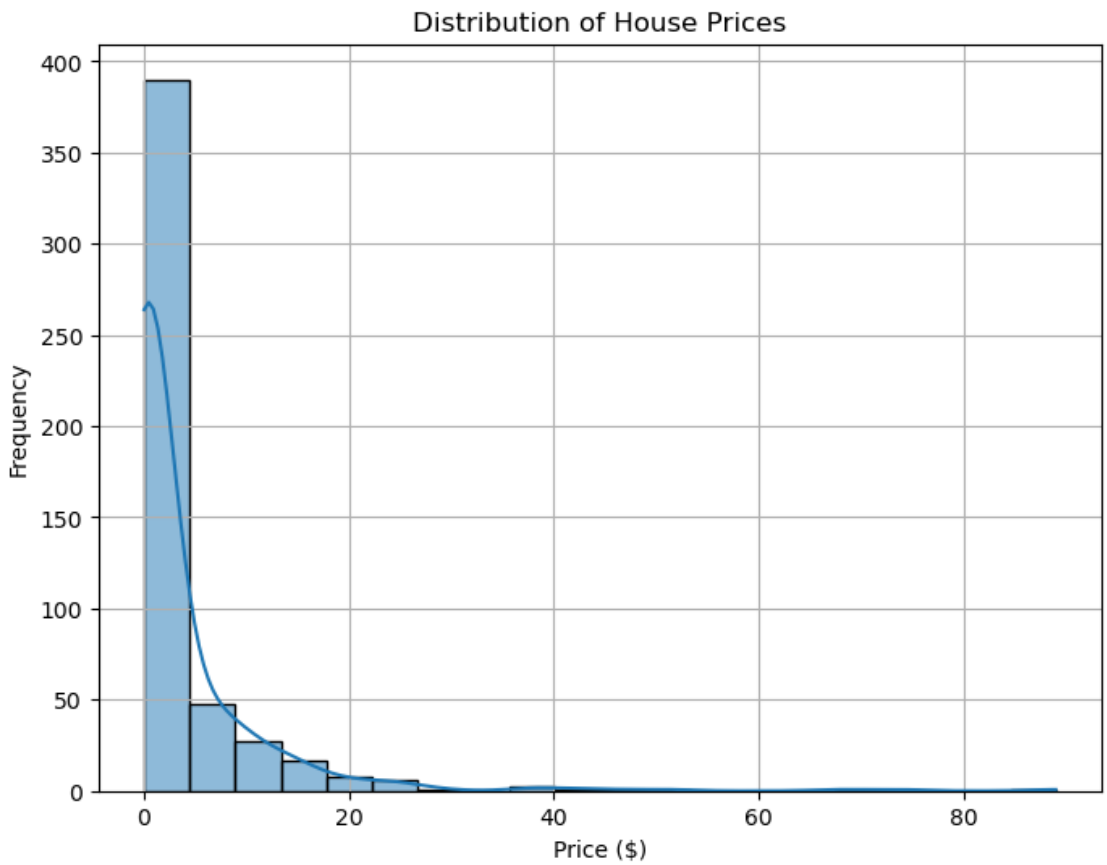
```
     6.5750_8.247  6.5750_8.259  6.5750_8.266  6.5750_8.297  6.5750_8.337  \
0           False         False         False         False         False
1           False         False         False         False         False
2           False         False         False         False         False
3           False         False         False         False         False
4           False         False         False         False         False
..            …             …             …             …             …
500         False         False         False         False         False
501         False         False         False         False         False
502         False         False         False         False         False
503         False         False         False         False         False
504         False         False         False         False         False

     6.5750_8.375  6.5750_8.398  6.5750_8.704  6.5750_8.725  6.5750_8.78
0           False         False         False         False        False
1           False         False         False         False        False
2           False         False         False         False        False
3           False         False         False         False        False
4           False         False         False         False        False
..            …             …             …             …            …
500         False         False         False         False        False
501         False         False         False         False        False
502         False         False         False         False        False
503         False         False         False         False        False
504         False         False         False         False        False

[505 rows x 458 columns]
```

[34]:
```python
from sklearn.preprocessing import LabelEncoder
label_encoder=LabelEncoder()
data['65.20=LabelEncoder'] = label_encoder.fit_transform(data['65.20'])
print("\nLabel Encoded Data:")
print(data)
```

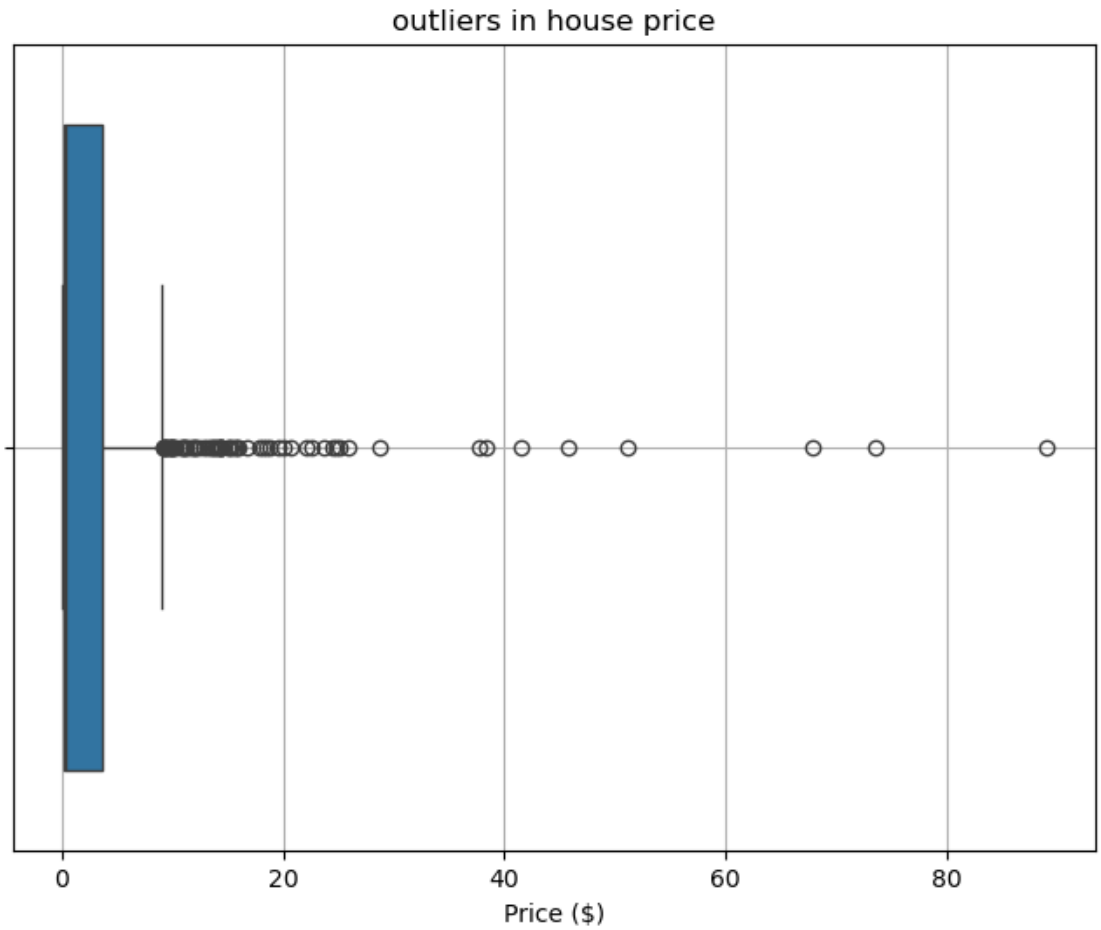
```
Label Encoded Data:
     0.00632  18.00  2.310  0  0.5380  6.5750  65.20  4.0900  1  296.0  15.30  \
0    0.02731    NaN   7.07  0   0.469   6.421   78.9  4.9671  2  242.0   17.8
1    0.02729    NaN   7.07  0   0.469   7.185   61.1  4.9671  2  242.0   17.8
2    0.03237    NaN   2.18  0   0.458   6.998   45.8  6.0622  3  222.0   18.7
3    0.06905    NaN   2.18  0   0.458   7.147   54.2  6.0622  3  222.0   18.7
4    0.02985    NaN   2.18  0   0.458   6.430   58.7  6.0622  3  222.0   18.7
..       …      …      …  ..      …       …      …       …  ..      …      …
500  0.06263    NaN  11.93  0   0.573   6.593   69.1  2.4786  1  273.0   21.0
501  0.04527    NaN  11.93  0   0.573   6.120   76.7  2.2875  1  273.0   21.0
502  0.06076    NaN  11.93  0   0.573   6.976   91.0  2.1675  1  273.0   21.0
503  0.10959    NaN  11.93  0   0.573   6.794   89.3  2.3889  1  273.0   21.0
```

```
504  0.04741    NaN  11.93  0   0.573   6.030   80.8  2.5050  1  273.0   21.0

     396.90  4.98  24.00  65.20=LabelEncoder
0    396.90  9.14   21.6                 225
1    392.83  4.03   34.7                 159
2    394.63  2.94   33.4                 112
3    396.90  5.33   36.2                 139
4    394.12  5.21   28.7                 153
..      …   …    …                   …
500  391.99  9.67   22.4                 187
501  396.90  9.08   20.6                 216
502  396.90  5.64   23.9                 288
503  393.45  6.48   22.0                 277
504  396.90  7.88   11.9                 231

[505 rows x 15 columns]
```

[4]:
```
plt.figure(figsize=(8,6))
sns.histplot(data['0.00632'],bins=20,kde=True)
plt.title("Distribution of House Prices")
plt.xlabel("Price ($)")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```



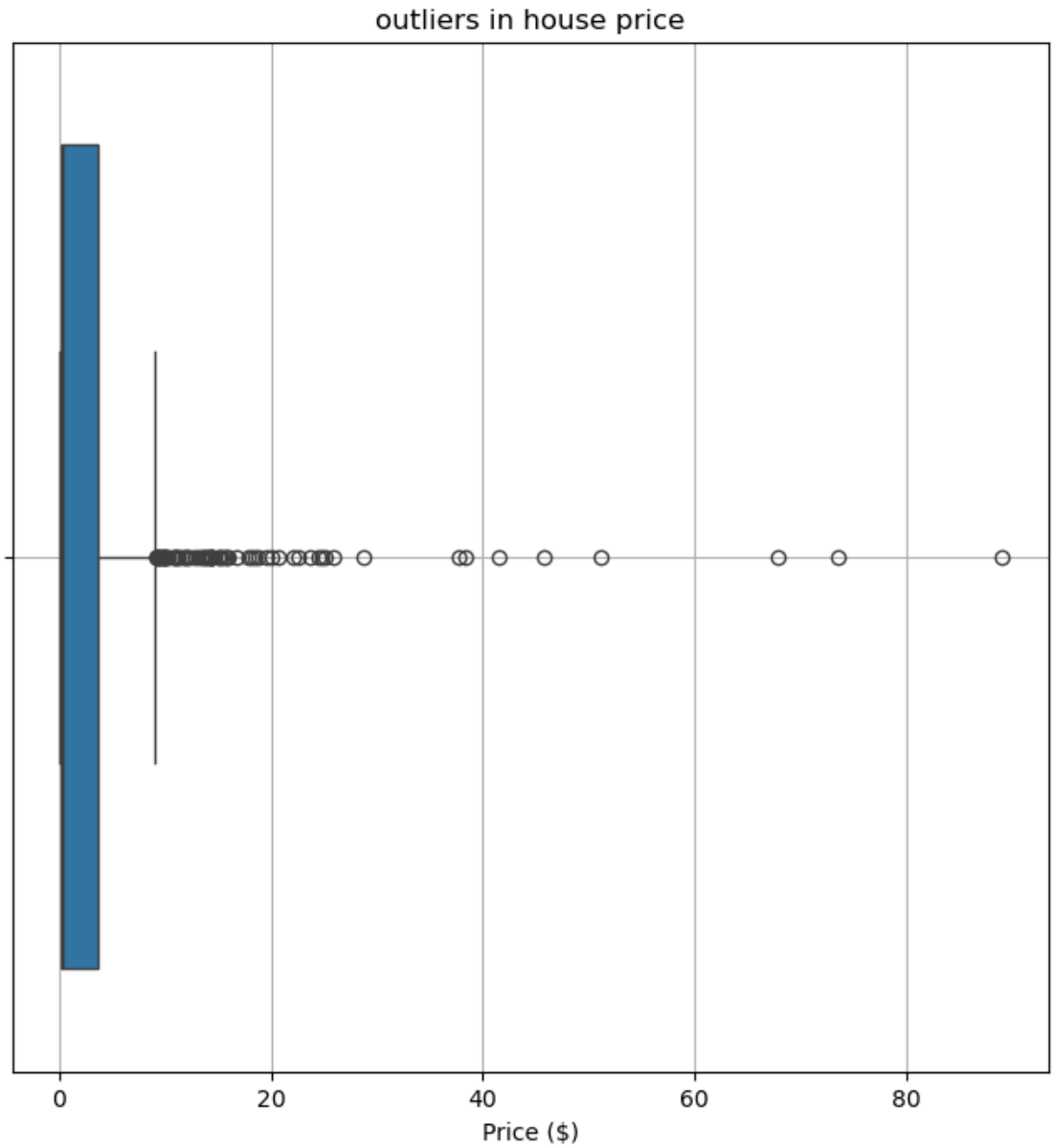
[5]:
```
plt.figure(figsize=(8,6))
sns.boxplot(data=data, x='0.00632')
plt.title("outliers in house price")
plt.xlabel("Price ($)")
plt.grid(True)
plt.show()
```



[23]:
```
#data cleaning
data = data[data['0.00632'] < 9000000]
```

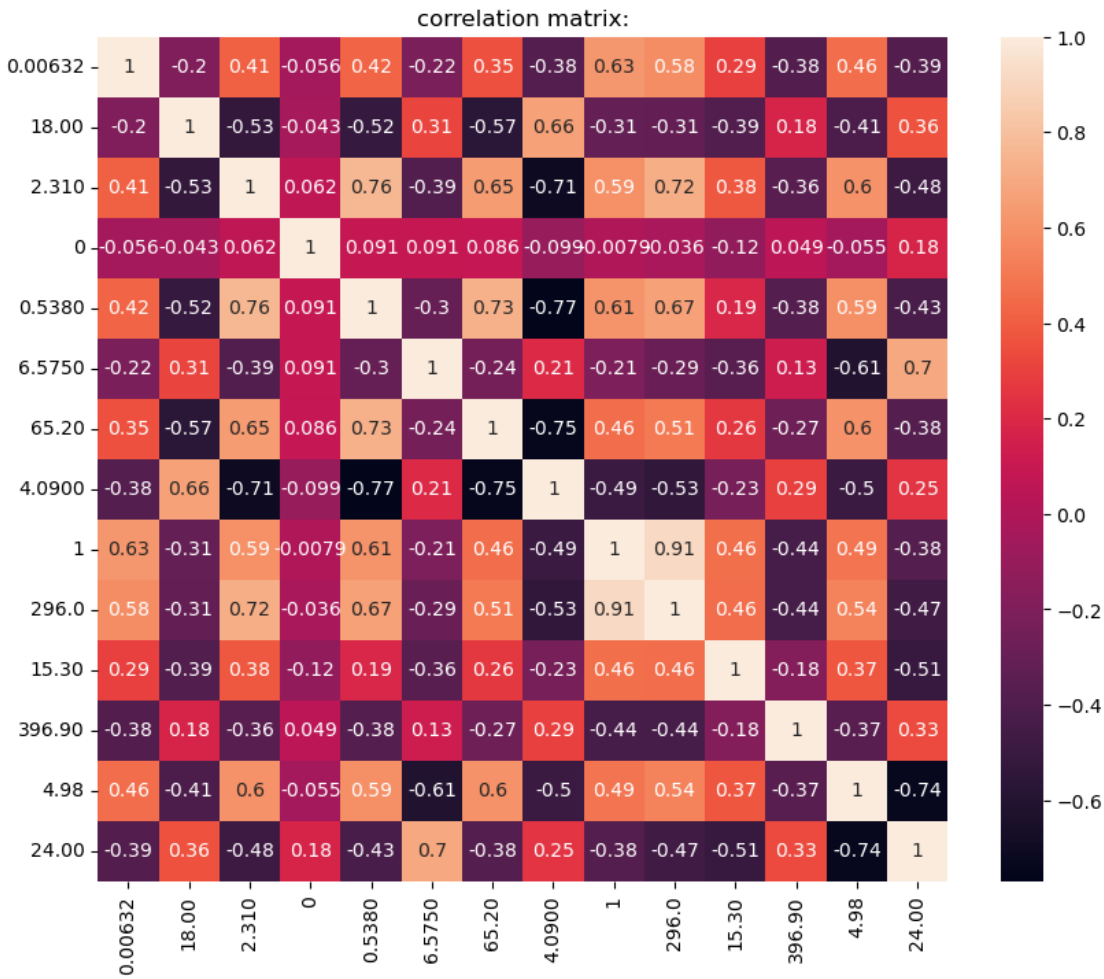
[ ]:
```
#visulaize outliers in house price
```

[24]:
```
plt.figure(figsize=(8,8))
sns.boxplot(data=data, x='0.00632')
plt.title("outliers in house price")
plt.xlabel("Price ($)")
plt.grid(True)
plt.show()
```

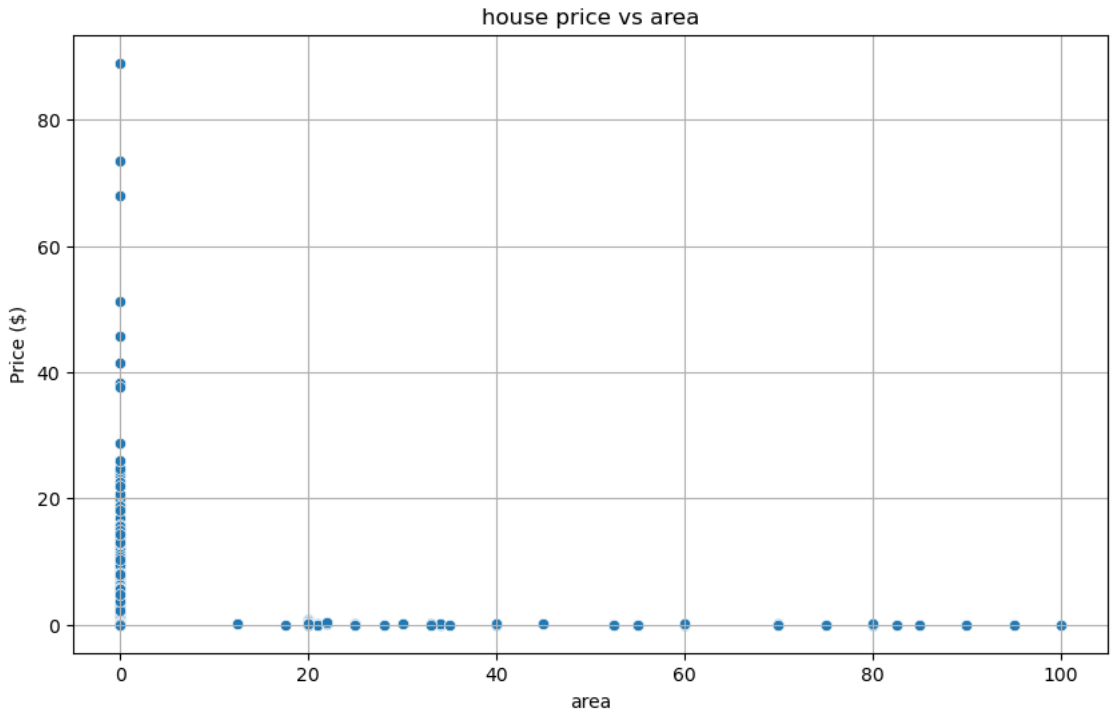


[25]:
```
#calculate correlation matrix
correlation_matrix=data.corr()
```

[30]:
```
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix, annot=True,)
plt.title("correlation matrix:")
plt.show()
```



[31]:
```
plt.figure(figsize=(10,6))
sns.scatterplot(data=data, x='18.00', y='0.00632')
plt.title("house price vs area")
plt.xlabel("area")
plt.ylabel("Price ($)")
plt.grid(True)
plt.show()
```


house price vs area
Price ($)
area
0
20
40
60
80
100

```
[ ]:
```